University of Pretoria ■ EBIT Faculty ■ **Department of Computer Science** ■ Academic Year **2023**

COS341 **Compiler** Construction ■ **Final Exam**

STUDENT	**RESULT**
First Name: Agape Nungo.	
Family Name: Mamphasa	
Stud.-Number: 18105882.	

This question-booklet must be filled in and returned together with the answer-booklet.

Internal Examiner: Prof. *S. Gruner*
External Examiner: Prof. *H. Suleman* (UCT)



Thursday the 15th of June 2023, 11:15h-14:15h in Hall "Centenary 5"

INSTRUCTIONS:
Read each question carefully **before writing** your answers *neatly* into the answer-booklet provided. Please avoid (as far as possible) to fill the answer-booklet with irrelevant rough-scribbles: instead, use your own rough-scribble pad for such purposes.
If the text of some question *explicitly demands* the application of a *specific* method, then you *must* apply the required method: in such cases you are not allowed to follow your own alternative method to solve the given problem "in your own way".
Wait until the invigilators give the start-signal before you begin to work.

FORBIDDEN:

- Any kind of academic dishonesty (as defined by the rule book of the University of Pretoria);
- Any electronic or computational devices such as portable computers, "smart" cell-phones, "smart" wrist-watches, e-book display devices (e.g.: "Kindle"), and the like;
- The course's textbook as a whole;
- Large-quantity bulk-prints of 3rd-party digital materials which you have not authored, (also including: the E-book version of our textbook, the PDF lecture slides, any internet wikipedia web-pages, any scientific papers from GoogleScholar, etc.);
- Direct prints of any JPG/GIF/photos taken with cameras from the lecturer's chalk-board.
- Writing with pencil that can be erased with a rubber-gum.

ALLOWED:

- *Student's own self-made* crib-notes in unlimited quanity (typed or written)

Timing Advice	MARKING SCHEME
Question **A**: *10 Minutes*	[5 Points]
Question **B**: *30 Minutes*	[10 Points]
Question **C**: *40 Minutes*	[10 Points]
Question **D**: *40 Minutes*	[10 Points]
Question **E**: *60 Minutes*	[15 Points]
	Total: [50 Points]

Question A (multiple choice): ***Related to various Chapters of the Book.*** **[5 Points]**

1. How can we decide whether *any* two different regular expressions are language-equivalent?

   Select ONLY ONE:

   a. We search for a word that is not contained in the intersection-language of both expressions
   b. We directly compare their non-deterministic finite automata
   c. We compare their minimized deterministic finite automata
   d. We build their parse trees and compare these trees

2. Why do we use context-free grammars in compiler construction?

   Select ONLY ONE:

   a. Because they are never ambiguous
   b. Because their ambiguity problem is decidable
   c. Because their word membership problem is decidable
   d. Because they are more expressive than context-sensitive grammars

3. Why do we prefer *LL1* parsers over Greibach Normal Form (*GNF*) parsers?

   Select ONLY ONE:

   a. Because the *LL1* class of languages is larger than the Greibach class of languages
   b. Because LL1 parsing is more efficient than *GNF* parsing
   c. Because not every context-free grammar can be brought into Greibach Normal Form
   d. Because no general parsing algorithm is known for grammars in *GNF*

4. How to we produce the final target code from intermediate code?

   Select ONLY ONE:

   a. We build a second parser on the basis of the intermediate language's own grammar
   b. We use a minimized deterministic finite automaton (*minDFA*)
   c. We use greedy string pattern matching (with backtracking)
   d. We use a search-tree as auxiliary data structure for finding the smallest programs

5. What happens during Liveness Analysis?

   Select ONLY ONE:

   a. The compiler interacts with the operating system's register allocator
   b. A dependency graph gets coloured with as many colours as we have registers
   c. A recursive system of set equations is solved to its fixpoint
   d. Dead-code is first flagged and then eliminated

Question B: ***Related to Chapter 2.*** **[10 Points]**

Given is the following small context-free grammar for branching-programs:

CFG for Branching Programs

1.	*PROG*	→	*INSTR* **;** *PROG*
2.	*PROG*	→	
3.	*INSTR*	→	**c**
4.	*INSTR*	→	*BRANCH*
5.	*BRANCH*	→	**i (b) {** *PROG* **}** *ELSE*
6.	*ELSE*	→	**e {** *PROG* **}**
7.	*ELSE*	→	

Comments:

c stands for some command, **i** stands for if, **b** stands for some boolean expresstion, **e** stands for else.

Turn the page →

Question A (multiple choice): ***Related to various Chapters of the Book.*** **[5 Points]**

1. How can we decide whether *any* two different regular expressions are language-equivalent?
   Select ONLY ONE:
   a. We search for a word that is not contained in the intersection-language of both expressions
   b. We directly compare their non-deterministic finite automata
   c. We compare their minimized deterministic finite automata
   d. We build their parse trees and compare these trees

2. Why do we use context-free grammars in compiler construction?
   Select ONLY ONE:
   a. Because they are never ambiguous
   b. Because their ambiguity problem is decidable
   c. Because their word membership problem is decidable
   d. Because they are more expressive than context-sensitive grammars

3. Why do we prefer *LL1* parsers over Greibach Normal Form (*GNF*) parsers?
   Select ONLY ONE:
   a. Because the *LL1* class of languages is larger than the Greibach class of languages
   b. Because LL1 parsing is more efficient than *GNF* parsing
   c. Because not every context-free grammar can be brought into Greibach Normal Form
   d. Because no general parsing algorithm is known for grammars in *GNF*

4. How to we produce the final target code from intermediate code?
   Select ONLY ONE:
   a. We build a second parser on the basis of the intermediate language's own grammar
   b. We use a minimized deterministic finite automaton (*minDFA*)
   c. We use greedy string pattern matching (with backtracking)
   d. We use a search-tree as auxiliary data structure for finding the smallest programs

5. What happens during Liveness Analysis?
   Select ONLY ONE:
   a. The compiler interacts with the operating system's register allocator
   b. A dependency graph gets coloured with as many colours as we have registers
   c. A recursive system of set equations is solved to its fixpoint
   d. Dead-code is first flagged and then eliminated

Question B: ***Related to Chapter 2.*** **[10 Points]**

Given is the following small context-free grammar for branching-programs:

CFG for Branching Programs

1.	*PROG*	→	*INSTR* **;** *PROG*
2.	*PROG*	→	
3.	*INSTR*	→	**c**
4.	*INSTR*	→	*BRANCH*
5.	*BRANCH*	→	**i (b) {** *PROG* **}** *ELSE*
6.	*ELSE*	→	**e {** *PROG* **}**
7.	*ELSE*	→	

Comments:
c stands for some command, **i** stands for if, **b** stands for some boolean expresstion, **e** stands for else.

Turn the page →

Investigate stepwise *whether* this grammar is *suitable* for **LL1**-Parsing :

- Do *not forget the additional rule* (which produces the End-of-File symbol **$**)
- Identify *those* of the given rules which are *relevant* for the investigation _____ [3 Points]
- For each *relevant* rule, *present* its **Look-Ahead-Set** (LA-Set) _____ [6 Points]
- *Conclude* the investigation by *comparing* the relevant LA-Sets _____ [1 Point]

Question C: Related to *Chapter 6*. **[10 Points]**

Given is a small source-code grammar for looping-programs as well as a small intermediate-code grammar for the language into which successfully parsed source-code programs shall be translated:

SOURCE Code Grammar:	**INTERMEDIATE Code Grammar**:
STATM → *STATM* **;** *STATM* *STATM* → **id :=** *EXPR* *STATM* → **while** *COND* **do** *STATM* *COND* → *EXPR* **relop** *EXPR* *EXPR* → **num** *EXPR* → **id** *EXPR* → *EXPR* **binop** *EXPR*	*ICPROGR* → *COMND* *ICPROGR* → *COMND ICPROGR* *COMND* → **label symbolicAddr** *COMND* → **id :=** *ATOM* *COMND* → **id := id binop** *ATOM* *COMND* → **goto symbolicAddr** *COMND* → **if id relop** *ATOM* **then symbolicAddr** **else symbolicAddr** *ATOM* → **id** *ATOM* → **num**

On this basis provide **translator functions** *TransStat* [6 Points], *TransCond* [1 Point] and *TransExp* [3 Points] by means of which *any* valid source code program, which conforms to the given source code grammar, can be correctly translated into its corresponding intermediate code program.

- Follow the method of the textbook, but *keep in mind that the examples given in the textbook differ* from the example given above: hence *you cannot simply copy&paste* the translator tables, un-adapted, out of the textbook into your answer booklet!
- You may **assume** that relevant *semantic information is available* in accessible *symbol tables*
- Hints: *TransStat* is using both *TransCond* and *TransExp*. *TransCond* is using only *TransExp*.

Question D: *Related to a combination of Chapter 6 and Chapter 9.* **[10 Points]**

Given is a small **source** program which contains a function call together with the function definition and body algorithm as follows:

MAIN Program's Algorithm	FUNCTION Definition with Body-Algorithm
x := 12 ; **y := 8 ;** **z :=** CALL **euclid(x , y)**	euclid (**a** , **b**) **while a ≠ b do** **c := b ;** **b := a mod b ;** **a := c ;** return **a**

Present the **final version** of the **intermediate code** into which *this given* source program (Main *and* Function) gets translated (including: code of the algorithms, prologue code, call-sequence, epilogue code) by means of the translation techniques which you have studied.

Turn the page →

Thereby you shall make the following **Assumptions**:

- The modulo operator **mod** is *included* in the intermediate language as a **binop**
- The not-equals check ≠ is *included* in the intermediate language as a **relop**
- The translation-strategy for functions shall be "caller-saves"
- *Enough frame-space is available* in a stack-frame for the function's three variables **a,b,c**
- *You can solve this question D without having solved the previous question C,* (simply by *applying* the translation techniques as they have been demonstrated in our course-module).

Further important Advice:

- As *stack management* in our lecture slides is somewhat different from stack management in our textbook, you are **free to choose which method** you want to apply, however you **must indicate your choice explicitly** (e.g.: *"Please note: I'm using the book method"*, or *"Please note: I'm using the the slides method"*) and then your work will be marked accordingly. MARKS WILL BE LOST **if** you make **no** indication at all, **or if** you give the **wrong** indication (for example if you say that you have chosen the "slides method" but your presentation is actually book method, or vice versa).
- **If** you are choosing the book method: For "caller-saves", remember the examples from Figures **9.2**, **9.3**, **9.4** but keep in mind that *you must adapt* those examples *specifically* to the program given above; you *cannot* simply *copy&paste* Figures 9.2, 9.3, 9.4 out of the book into your answer-booklet without adaptation to the given scenario of above.

---

in[i] = gen[i] ∪ (out[i] \ kill[i])
out[i] = ∪ in[j]

Question E: ***Related to Chapter 8.*** **[15 Points]**

Given is the following line-numbered program ***in intermediate code*** (which repeatedly divides an even positive number by 2 until the result of the division yields an odd number):

```
1.   n := 0
2.   t := 2
3.   x := u      // comment: assume that the value of u is coming from some user-input elsewhere
4.   IF x==n GOTO 9      // comment: prevent infinite looping on the even number null
5.   b := x mod t        // comment: b is used below for a boolean comparison
6.   IF b ≠ n GOTO 9     // comment: end the program if the number is odd
7.   x := x div t        // comment: divide the even number by 2
8.   GOTO 5              // comment: repeat the loop
9.   return x            // comment: end of program
```

You see that there are *five variables*: **n**, **t**, **x**, **u**, **b**.

E.1) [10 Points]

Carry out liveness analysis for the given program. Do the intermediate steps on some rough scribble pad, and **present only the final results** –*AFTER the fixpoint of the analysis has been reached*– in a **table** of the following format:

Line number *i*	**succ**[*i*]	**gen**[*i*]	**kill**[*i*]	**out**[*i*]	**in**[*i*]

E.2) [5 Points]

Use your analysis results from sub-question E.1 to **present the program's variable dependencies** in a **table** of the following format:

Relevant Line number *i*	Left-hand-side variable	Interferes with